

# AutoBild 爬蟲系統 v10.0

## 程式碼審查與問題診斷報告

報告日期：2026 年 7 月 21 日

分析方式：實際抓取 AutoBild 網頁 DOM 結構比對

### 問題總覽

致命問題 (會導致大量資料遺失)：共 4 項

嚴重問題 (會導致部分資料錯誤)：共 3 項

一般問題 (影響穩定性或完整性)：共 5 項

本報告基於實際抓取 AutoBild 網站 HTML 並分析真實 DOM 結構後，逐一比對程式碼中的選擇器、邏輯與流程，找出所有會導致資料搜不到的問題。

## 一 致命問題 (會導致大量資料遺失)

### [致命] 問題 1: Model 連結比對邏輯永遠失敗

程式碼使用 `h.startswith(b_url)` 來判斷某個 `href` 是否為當前品牌的子車款。但 `b_url` 是完整網域 URL (如 `https://www.autobild.de/marken-modelle/audi/`)，而 HTML 中的 `href` 屬性通常是相對路徑 (如 `/marken-modelle/audi/a1/`)。相對路徑永遠不會 `startswith` 完整 URL，導致幾乎所有 model 都被過濾掉。

問題程式碼：

```
# b_url = "https://www.autobild.de/marken-modelle/audi/"
# h = "/marken-modelle/audi/a1/"
if h and h.count("/") > b_url.count("/") and h.startswith(b_url):
    # ↑ 相對路徑永遠不會 startswith 完整 URL，此條件永假
```

建議修正：

```
# 修正方案：先將相對路徑轉為完整 URL 再比對
from urllib.parse import urljoin
full_h = urljoin(Config.BASE_URL, h)
if full_h.startswith(b_url.rstrip("/")):
    models.append(full_h)
```

影響：几乎所有品牌的子車款都无法被识别，导致每个品牌只抓到 model 首页而非各变体

### [致命] 問題 2: 「ALLE MODELLE」按鈕文字比對過嚴

程式碼使用 `el.innerText.trim().toUpperCase() === 'ALLE MODELLE'` 來尋找按鈕。但實際網頁上的按鈕文字是 'Alle Modelle' (含前後空格)，且 `toUpperCase()` 轉換後為 'ALLE MODELLE'。表面上看似正確，但 `innerText` 在某些瀏覽器中可能包含隱藏子元素的文字，導致比對失敗。更關鍵的是：即使找到了按鈕，它是 Vue.js 控制的 `<button>` 元素，點擊後可能不會觸發傳統的頁面導航。

問題程式碼：

```
# 實際按鈕 HTML:
# <button class="" data-v-b8383ef5> Alle Modelle </button>
# 文字含前後空格，且是 Vue button 不是 <a> 連結
const btns = Array.from(document.querySelectorAll('a, button, li, span'))
    .filter(el => el.innerText && el.innerText.trim().toUpperCase() === 'ALLE MODELLE');
if(btns.length > 0) btns[0].click();
```

建議修正：

```
# 更穩健的比對方式 (忽略多餘空白) :
const btns = Array.from(document.querySelectorAll('button'))
    .filter(el => el.textContent &&
        el.textContent.replace(/\s+/g, ' ').trim().toUpperCase() === 'ALLE MODELLE');
if(btns.length > 0) btns[0].click();
```

影響：如果按鈕找不到，就無法顯示隱藏的 legacy 車款 (如 Audi 50、80、90 等28款)

### [致命] 問題 3: 「WEITERE VARIANTEN ANZEIGEN」按鈕文字不存在

程式碼在                      while                      迴圈中尋找包含                      'WEITERE                      VARIANTEN                      ANZEIGEN'

文字的按鈕。但實際網頁上這個文字根本不存在。展開按鈕的實際 class 是 vvp\_fuelType-dataFooterToggleButton，且只包含一個 SVG 箭頭圖示，沒有任何文字。因此 while 迴圈第一次就找不到按鈕，直接 break，導致需要展開的隱藏變體永遠不會被載入。

#### 問題程式碼：

```
# 程式碼尋找的文字：
'WEITERE VARIANTEN ANZEIGEN'

# 實際按鈕結構（無文字，只有 SVG 箭頭）：
# <div class="vvp_fuelType-dataFooterToggleButton">
#   <svg class="vvp_fuelType-dataFooterToggleButtonArrow">...</svg>
# </div>
```

#### 建議修正：

```
# 修正：使用 class 選擇器而非文字
while (true) {
  const clicked = await page.evaluate() => {
    const btn = document.querySelector(
      '.vvp_fuelType-dataFooterToggleButton');
    if (btn && btn.offsetParent !== null) {
      btn.click();
      return true;
    }
    return false;
  };
  if (!clicked) break;
  await async.sleep(1.2);
}
```

影響：所有需要展開的隱藏變體列都不会被載入，資料嚴重不完整

### [致命] 問題 4：車款詳細頁面 HSN/TSN 等欄位根本不存在

程式碼在詳細頁面中透過搜尋 'Basisdaten'、'HSN/TSN'、'Karosserieform'、'Kraftstoffart' 等文字來挖取資料。但實際網頁上這些標籤文字根本不存在於 DOM 中。HSN、TSN、Karosserieform 完全不出現在頁面上。Kraftstoffart 只出現在計數標籤中（如 '4 Kraftstoffarten'），而非資料表格的欄位標題。因此所有欄位都會是 N/A。

#### 問題程式碼：

```
# 程式碼搜尋的標籤：
txt === 'hsn/tsn schlüsselnummern' → 找不到
txt === 'karosserieform' → 找不到
txt === 'kraftstoffart' → 找不到（只有 "4 Kraftstoffarten"）
txt === 'bauzeitraum' → 找不到
txt === 'modell' → 找不到
```

#### 建議修正：

```
# 正確做法：直接解析 vike_pageContext JSON
const jsonData = await page.evaluate() => {
  const el = document.querySelector('#vike_pageContext');
  return el ? JSON.parse(el.textContent) : null;
}
```

```
});  
// jsonData.irContent.vehicleVariants 包含所有結構化資料：  
// { fuelType, name, name2, power, price,  
#   sale_start, sale_end, acceleration, ... }
```

影響：所有詳細欄位（HSN/TSN、車身類型、燃料類型、年份）都是 N/A，資料完全無用

## 一 嚴重問題（曾導致部分資料錯誤）

### [嚴重] 問題 5：車款變體 URL 收集使用不存在的選擇器

程式碼使用 `.vvp_fuelType-dataBodyRow` 來選取每一列變體，然後從中找 `<a>` 標籤取得 `href`。雖然 `.vvp_fuelType-dataBodyRow` 確實存在（實測找到 59 個），但每列的結構是 `div.vvp_fuelType-dataBodyRowCol`，裡面是純文字（車名、馬力、價格），並沒有 `<a>` 標籤。因此 `vehicle_hrefs` 永遠是空陣列，fallback 到只抓 model 首頁。

問題程式碼：

```
# 程式碼：
return Array.from(document.querySelectorAll(
  '.vvp_fuelType-dataBodyRow'))
  .map(row => {
    const a = row.querySelector('a'); // ← 永遠是 null!
    return a ? a.getAttribute('href') : null;
  }).filter(h => h);

# 實際結構（無 <a> 標籤）：
# <div class="vvp_fuelType-dataBodyRow">
#   <div class="vvp_fuelType-dataBodyRowCol">Golf 1.0 TSI</div>
#   <div class="vvp_fuelType-dataBodyRowCol">90 PS</div>
#   <div class="vvp_fuelType-dataBodyRowCol">21.765 EUR</div>
# </div>
```

建議修正：

```
# 修正方案 A：從 JSON 取得所有變體資料（推薦）
const data = await page.evaluate() => {
  const el = document.querySelector('#vike_pageContext');
  if (!el) return [];
  const ctx = JSON.parse(el.textContent);
  const variants = ctx.irContent?.vehicleVariants || [];
  return variants.flatMap(vt =>
    vt.fuelType.flatMap(ft => ft.data)
  );
};

# 修正方案 B：從歷史資料表(vv_)中找連結
const links = await page.evaluate() => {
  return Array.from(document.querySelectorAll(
    '.vv_fuelType-dataBodyLineLink a, ' +
    '.vvp_fuelType-dataBodyRowCol a'))
    .map(a => a.getAttribute('href'))
    .filter(h => h);
};
```

影響：所有變體 URL 都抓不到，每個 model 只會 fallback 到首頁，完全沒有變體資料

### [嚴重] 問題 6：Dom 挖字邏輯假設相鄰元素是值

程式碼遍歷所有 `div/span/td/th`，假設 `allCells[j]` 是標籤、`allCells[j+1]` 就是值。但實際 DOM 中標籤和值不一定相鄰，中間可能夾著其他元素（如 `class` 包裝層、SVG 圖示等）。此外，由於問題 4 中這些標籤根本不存在，這段邏輯即使修正了也無法正確運作。

問題程式碼：

```
let val = allCells[j+1] ? allCells[j+1].innerText.trim() : '';
# 問題: allCells[j+1] 可能是同層的下一個無關元素
# 例如 <div><span>Label</span><span class="icon">...</span><span>Value</span></div>
# j+1 拿到的是 icon 而非 Value
```

建議修正：

```
# 建議: 放棄 DOM 挖字，改用 JSON 結構化資料
# vike_pageContext 中的資料已經是結構化的，無需解析 DOM
```

影響：即使標籤存在，挖到的值也可能是錯誤的或空的

### [嚴重] 問題 7: HSN\_TSN 拆分邏輯不完整

程式碼只考慮用逗號分隔的 HSN/TSN。但實際上 HSN/TSN 可能用空格、換行、分號等分隔。此外，由於問題 4 導致 HSN\_TSN 永遠是 N/A，這段拆分邏輯根本不會被執行。

問題程式碼：

```
if ',' in final_hsn_tsn:
    for code in [c.strip() for c in final_hsn_tsn.split(',')]:
        # 只處理逗號分隔
```

建議修正：

```
# 建議: 支援多種分隔符
import re
codes = re.split(r'[,\n\r]+', final_hsn_tsn)
codes = [c.strip() for c in codes if c.strip()]
```

影響：多組 HSN/TSN 編碼無法被正確拆分寫入

## 三 一般問題（影響穩定性或完整性）

### [一般] 問題 8：品牌 URL 尾部斜線過濾過嚴

程式碼要求 `href` 必須以 `/` 結尾才收錄。雖然實測 AutoBild 的品牌 URL 都有尾部斜線，但如果未來網站改版移除尾部斜線，就會遺漏品牌。

問題程式碼：

```
if full.endswith("/") and full not in seen:
    seen.add(full)
    brand_urls.append(full)
```

建議修正：

```
# 建議：統一加上尾部斜線
full = full.rstrip("/") + "/"
if full not in seen:
    seen.add(full)
    brand_urls.append(full)
```

影響：目前影響小，但缺乏彈性

### [一般] 問題 9：Cookie 同意只處理一次

程式碼只在第一個頁面處理 Cookie 彈窗。如果換到新 `brand/model` 頁面時彈窗再次出現（或 `iframe` `id` 變化），會阻擋後續所有互動操作。

問題程式碼：

```
# 只在開頭處理一次
iframe = page.frame_locator('iframe[id^="sp_message_iframe"]')
await iframe.get_by_role("button", name="Alle akzeptieren").click(timeout=8000)
```

建議修正：

```
# 建議：在每個新頁面都嘗試處理
async def dismiss_cookie(page):
    try:
        iframe = page.frame_locator('iframe[id^="sp_message_iframe"]')
        btn = iframe.get_by_role("button", name="Alle akzeptieren")
        await btn.click(timeout=5000)
    except:
        pass
```

影響：後續頁面可能被 Cookie 彈窗阻擋，導致點擊操作失敗

### [一般] 問題 10：brand 頁面連結選擇器過於寬泛

程式碼使用 `a[href*='/marken-modelle/']` 來選取所有連結，然後再過濾。這個選擇器會匹配到大量非導航連結（如圖片連結、廣告連結等），增加不必要的處理時間。

問題程式碼：

```
links = await page.query_selector_all("a[href*='/marken-modelle/']")
# 會匹配到所有包含 /marken-modelle/ 的連結
```

建議修正：

```
# 建議：使用更精確的選擇器
# 方案 A：直接從 JSON-LD 取得所有 model URL
model_urls = await page.evaluate("() => {
  const scripts = document.querySelectorAll(
    'script[type="application/ld+json"]');
  for (const s of scripts) {
    const data = JSON.parse(s.textContent);
    if (data['@type'] === 'ItemList') {
      return data.itemListElement.map(
        i => i.item.url);
    }
  }
  return [];
}");

# 方案 B：使用 modelTeaser_link 選擇器
links = await page.query_selector_all(".modelTeaser_link")
```

影響：處理時間增加，且可能誤收非導航連結

#### [一般] 問題 11: model 頁面選擇器過於寬泛

程式碼使用 `a[href*='/marken-modelle/']` 並透過 `h.startswith(b_url)` 來過濾。除了前述的 `startswith` 問題外，這個選擇器也會匹配到側邊欄、頁尾等非主要導航區域的連結。

問題程式碼：

```
model_links = await page.query_selector_all("a[href*='/marken-modelle/']")
# 匹配範圍過大
```

建議修正：

```
# 建議：使用品牌頁面的 modelTeaser 選擇器
model_links = await page.query_selector_all(".modelTeaser_link")
# 或從 JSON-LD ItemList 取得
```

影響：可能收到重複或非主要導航的 model 連結

#### [一般] 問題 12: 缺少錯誤重試機制

程式碼在 `detail_page.goto` 失敗時直接 `except pass` 跳過，沒有重試機制。網路不穩定時會永久丟失該車款資料。

問題程式碼：

```
try:
  await detail_page.goto(detail_url, timeout=45000, ...)
  # ... 處理資料
except Exception as e:
  pass # 直接跳過，永不重試
```

建議修正：



```
# 建議：加入重試機制
MAX_RETRIES = 3
for attempt in range(MAX_RETRIES):
    try:
        await detail_page.goto(detail_url, timeout=45000, ...)
        break
    except Exception as e:
        if attempt == MAX_RETRIES - 1:
            print(f" [失敗] {detail_url}: {e}")
        else:
            await asyncio.sleep(2 ** attempt)
```

影響：網路不穩定時會永久丟失車款資料

## 四 最佳實踐建議

### 建議 1：直接使用 vike\_pageContext JSON

AutoBild 的每個車款詳細頁面都包含一個 `<script id="vike_pageContext" type="application/json">` 標籤，裡面有完整的結構化資料（車名、馬力、價格、生產年份、燃料類型等）。與其花力氣在 DOM 中挖字，不如直接解析這個 JSON，資料更準確且完整。

### 建議 2：使用 JSON-LD 取得 model 列表

每個品牌頁面的 JSON-LD ItemList 包含該品牌所有 model 的名稱和 URL（共 45 個 Audi model），包括隱藏的 legacy 車款。這比點擊「Alle Modelle」按鈕再抓取連結更可靠。

### 建議 3：使用精確的 CSS 選擇器

品牌頁面：使用 `.modelTeaser_link` 取得 model 連結。

車款頁面：使用 `.vw_fuelType-dataBodyLineLink a` 取得變體詳細頁面連結。

展開按鈕：使用 `.vvp_fuelType-dataFooterToggleButton` 而非文字比對。

### 建議 4：加入請求速率限制與重試

每個請求之間加入隨機延遲（已有），但建議在失敗時加入指數退避重試（exponential backoff），避免因暫時性錯誤而永久丟失資料。

### 建議 5：使用 headless=False 除錯

在開發階段使用

headless=False

開啟瀏覽器視窗，可以即時觀察爬蟲的行為，快速發現選擇器不匹配、按鈕點擊失敗等問題。

## 附錄 AutoBild 實際 DOM 結構參考

### A. 品牌頁面 (如 /marken-modelle/audi/)

```
<div class="brandPage__modelsSelector">
  <button>Aktuelle Modelle</button>
  <button>Elektro Modelle</button>
  <button> Alle Modelle </button> ← 實際按鈕 (含前後空格)
</div>

<div class="modelTeasers__grid">
  <div class="modelTeaser">
    <a class="modelTeaser__link"
      href="https://www.autobild.de/marken-modelle/audi/a1/">
      <p class="modelTeaser__title">A1</p>
      <span class="modelTeaser__priceValue">23.300 EUR</span>
    </a>
  </div>
  ... 共 45 個 model (17 個可見 + 28 個隱藏)
</div>
```

### B. 車款詳細頁面 (如 /marken-modelle/vw/golf/)

```
<!-- 歷史技術資料表 (vv__ 系統) -->
<div class="vv__fuelType-dataBodyLine">
  <div class="boost">Golf 1.0 TSI OPF
    <span>05/2020 -> 06/2022</span>
  </div>
  <div>90 PS</div>
  <div>11.9 s</div>
  <div>5.3 l/100 km</div>
  <div>21.765 EUR</div>
  <div class="vv__fuelType-dataBodyLineLink">
    <a href="/marken-modelle/vw/golf/...">...</a>
  </div>
</div>

<!-- 當前價格表 (vvp__ 系統) -->
<div class="vvp__fuelType-dataBodyRow">
  <div class="vvp__fuelType-dataBodyRowCol">
    <span class="boost">Golf 1.5 TSI OPF, Life</span>
  </div>
  <div class="vvp__fuelType-dataBodyRowCol">116 PS</div>
  <div class="vvp__fuelType-dataBodyRowCol">32.080 EUR</div>
  <!-- 注意: 此處無 <a> 標籤! -->
```

### C. vike\_pageContext JSON 結構 (推薦使用)

```
<script id="vike_pageContext" type="application/json">
{
  "irContent": {
    "vehicleVariants": [
```

```

{
  "constructionType": "Fließheck",
  "fuelType": [
    {
      "fuelType": "benzin",
      "totalCount": 67,
      "data": [
        {
          "id": "6718ec5cb697918e9c386c39",
          "name": "Golf 1.0 TSI OPF",
          "name2": null,
          "power": 90,
          "price": 21765,
          "acceleration": 11.9,
          "consumption_fuel": 5.3,
          "sale_start": "202005",
          "sale_end": "202206"
        },
        ... 共 67 個 Benzin 變體
      ]
    },
    { "fuelType": "diesel", ... },
    { "fuelType": "benzin-elektro-plugIn", ... }
  ]
}
]
}
}
</script>

```

#### D. 展開按鈕的實際結構

```

<!-- 無文字, 只有 SVG 箭頭 -->
<div class="vvp__fuelType-dataFooter">
  <div class="vvp__fuelType-dataFooterToggle">
    <div class="vvp__fuelType-dataFooterToggleButton">
      <svg class="vvp__fuelType-dataFooterToggleButtonArrow">
        <!-- 向下箭頭 SVG -->
      </svg>
    </div>
  </div>
</div>

```